

Theoretische Informatik II

Markus Hecher

Universität Potsdam, Theoretische Informatik, SoSe 2026

Datum: 2026-06-08

Name:**Matrikelnummer:****Aufgabe 1 (Wahr-Falsch Fragen)**

Kreuzen Sie bei folgenden Aussagen jeweils an, ob diese zutreffen (“Ja”) oder nicht zutreffen (“Nein”). Lesen Sie dafür die Aussage bitte *sorgfältig durch*!

Ja Nein

- ☒ ☒ Es gibt keine totale Funktion, die man nicht mit primitiv rekursiven Funktionen alleine abbilden kann.
- ☒ ☒ Total berechenbare Funktionen kann man nicht mit einem Berechenbarkeitskonzept umfassen.
- ☒ ☒ Die Church-Turing These ist weitgehend anerkannt und besagt, dass intuitiv berechenbare Funktionen mit Turing-berechenbarkeit zusammenfallen.
- ☒ ☒ Die unbeschränkte Minimierung wird nur für die Berechnung via nicht totaler Funktionen benötigt.
- ☒ ☒ Unbeschränkte Minimierung kann man mit Loop-Programmen simulieren.
- ☒ ☒ Primitiv-rekursive Funktionen mit unbeschränkter Minimierung sind gleich mächtig wie Turingmaschinen.
- ☒ ☒ Beinahe alle Funktionen kann man mit primitiv-rekursiven bzw. Loop-Programmen abbilden (verbleibende Funktionen wachsen extrem stark).
- ☒ ☒ Das Problem bei der Berechnung der Ackermannfunktion ist die unbeschränkte Stacktiefe, die dafür sorgt, dass die Funktion nicht mehr total ist.
- ☒ ☒ Es gibt unendlich viele totale Funktionen, die man nicht mit Loop-Programmen abbilden kann.
- ☒ ☒ Mit geeigneten Reduktionen kann man Unlösbarkeit nachweisen.
- ☒ ☒ Die Cantorsche Paarungsfunktion angewendet auf eine Zahl $n \in \mathbb{N}$ entspricht dieser Zahl: $\langle n \rangle = n$.
- ☒ ☒ Die Cantorsche Paarungsfunktion und zugehörige Umkehrfunktionen sind primitiv-rekursiv.
- ☒ ☒ Mehrstellige Funktionen sind in der Berechenbarkeitstheorie mächtiger als einstellige.
- ☒ ☒ Bei Turingmaschinen ist das Trennzeichen (#) im Bandalphabet unbedingt notwendig und kann daher nicht mit Bandalphabet ohne Trennzeichen simuliert werden.
- ☒ ☒ Das t-Halteproblem, welches fragt, ob eine Turingmaschine in t Schritten hält, ist entscheidbar.
- ☒ ☒ Von keinem Programm kann automatisch entschieden werden, ob es auf einer bestimmten Eingabe terminiert.

(8 Punkte)**Aufgabe 2 (Potenzfunktion)**

Definieren Sie direkt mittels primitiver Rekursion die Potenzfunktion **pow** (Beispiel: $\text{pow}(2, 23) = 2^{23}$). Tipp: Sie dürfen die Funktion **mul** verwenden.

(4 Punkte)

Lösung: $\text{pow} : \text{Pr}[\text{const}_1^1, \text{mul} \circ (\text{proj}_1^3, \text{proj}_3^3)]$

Aufgabe 3 (Minimierung)

Zeigen Sie mittels Minimierung, dass der Logarithmus zur Basis b (aufgerundet auf die nächste ganze Zahl) primitiv-rekursiv ist (Beispiel: $\log(2^{11} + 7, 2) = 12$). Tipp: Verwenden sie pow und Vergleichsoperatoren aus der VL! **(5 Punkte)**

Lösung: $\text{isGtExp} : \geq \circ (\text{pow} \circ (\text{proj}_2^3, \text{proj}_3^3), \text{proj}_1^3)$
 $\log : \text{Mn}[\text{isGtExp}] \circ (\text{proj}_1^2, \text{proj}_2^2, \text{proj}_1^2)$

Aufgabe 4 (Cantorsche Paarungsfunktion)

Zeigen Sie, dass die Funktion zip , die je 2 Sequenzen $p1, p2$ mit Hilfe einer vorgegebenen Länge n miteinander verzahnt, primitiv rekursiv ist. Beispiel: $p1 = \langle 1, \langle 2, \langle 3, 4 \rangle \rangle \rangle$, $p2 = \langle 2, \langle 6, \langle 5, 7 \rangle \rangle \rangle$, $n = 4$, daher $\text{zip}(p1, p2, n) = \langle \langle 1, 2 \rangle, \langle \langle 2, 6 \rangle, \langle \langle 3, 5 \rangle, \langle 4, 7 \rangle \rangle \rangle \rangle$. Gehen Sie dabei schrittweise wie folgt vor. **(8 Punkte)**

a) Definieren Sie eine Hilfsfunktion firstzip , die die ersten beiden Zahlen in $p1, p2$ verzahnt (Beispiel: $\text{firstzip}(p1, p2) = \langle 1, 2 \rangle$):

Lösung: $\text{firstzip} : \langle \rangle \circ (\pi_1^2 \circ \text{proj}_1^2, \pi_1^2 \circ \text{proj}_2^2)$

b) Definieren Sie die Funktion izip , welches die i -ten Zahlen von $p1, p2$, verzahnt (Beispiele: $\text{izip}(p1, p2, 1) = \text{firstzip}(p1, p2)$, $\text{izip}(p1, p2, 2) = \langle 2, 6 \rangle$, $\text{izip}(p1, p2, 3) = \langle 3, 5 \rangle$, $\text{izip}(p1, p2, 4) = \langle 4, 7 \rangle$). Tipp: Verwenden Sie firstzip und die bekannte Funktion $\text{tail}(p, n)$, welche n mal hintereinander π_2^2 auf p aufruft (Beispiele: $\text{tail}(p1, 0) = p1$, $\text{tail}(p1, 2) = \langle 3, 4 \rangle$):

Lösung: $\text{izip} : \text{firstzip} \circ (\text{tail} \circ (\text{proj}_1^3, \text{pred} \circ \text{proj}_3^3), \text{tail} \circ (\text{proj}_2^3, \text{pred} \circ \text{proj}_3^3))$

c) Verwenden Sie izip , um direkt den Schrittfall mit Hilfe der Funktion reczip zu definieren (Beispiele: $\text{reczip}(p1, p2, 3, \langle 4, 7 \rangle) = \langle \langle 3, 5 \rangle, \langle 4, 7 \rangle \rangle$, $\text{reczip}(p1, p2, 2, \langle \langle 3, 5 \rangle, \langle 4, 7 \rangle \rangle) = \langle \langle 2, 6 \rangle, \langle \langle 3, 5 \rangle, \langle 4, 7 \rangle \rangle \rangle$):

Lösung: $\text{reczip} : \langle \rangle \circ (\text{izip} \circ (\text{proj}_1^4, \text{proj}_2^4, \text{proj}_3^4), \text{proj}_4^4)$

d) Fügen Sie alles via primitiver Rekursion (Pr) zusammen (Basisfall direkt via `izip`, Schrittfall direkt via `reczip`). Erstellen Sie dafür `zipAt`, welches im Basisfall 3 Argumente $p1, p2, n$ bekommt und im Schrittfall 5 Argumente $p1, p2, n, m, (\text{zipAt}(p1, p2, n, m))$, sodass $\text{zip} = \text{zipAt} \circ (\text{proj}_1^3, \text{proj}_2^3, \text{proj}_3^3, \text{pred} \circ \text{proj}_3^3)$:

Lösung: $\text{zipAt} : \text{Pr} [\text{izip} \circ (\text{proj}_1^3, \text{proj}_2^3, \text{proj}_3^3), \text{reczip} \circ (\text{proj}_1^5, \text{proj}_2^5, \text{sub} \circ (\text{pred} \circ \text{proj}_3^5, \text{proj}_4^5), \text{proj}_5^5)]$